

from practical training inefficiencies. Token-granular selection strategy used in approaches like HashAttention (Desai et al., 2024) leads to the need to load a large number of individual tokens from the KV cache during attention computation. This non-contiguous memory access prevents efficient adaptation of fast attention techniques like FlashAttention, which rely on contiguous memory access and blockwise computation to achieve high throughput. As a result, implementations are forced to fall back to low hardware utilization, significantly degrading training efficiency.

2.3. Native Sparsity as an Imperative

These limitations in inference efficiency and training viability motivate our fundamental redesign of sparse attention mechanisms. We propose NSA, a natively sparse attention framework that addresses both computational efficiency and training requirements. In the following sections, we detail the algorithmic design and operator implementation of NSA.

3. Methodology

Our technical approach spans algorithm design and kernel optimization. In the following subsections, we first introduce the background of our methodology. Then we present the overall framework of NSA, followed by its key algorithmic components. Finally, we detail our hardware-optimized kernel design that maximizes practical efficiency.

3.1. Background

Attention Mechanism is widely used in language modeling where each query token $\mathbf{q}_t$ computes relevance scores against all preceding keys $\mathbf{k}_{:t}$ to generate a weighted sum of values $\mathbf{v}_{:t}$. Formally, for an input sequence of length t , the attention operation is defined as:

$$\mathbf{o}_t = \text{Attn}(\mathbf{q}_t, \mathbf{k}_{:t}, \mathbf{v}_{:t}) \quad (1)$$

where Attn denotes the attention function:

$$\text{Attn}(\mathbf{q}_t, \mathbf{k}_{:t}, \mathbf{v}_{:t}) = \sum_{i=1}^t \frac{\alpha_{t,i} \mathbf{v}_i}{\sum_{j=1}^t \alpha_{t,j}}, \quad \alpha_{t,i} = e^{\frac{\mathbf{q}_t^\top \mathbf{k}_i}{\sqrt{d_k}}}. \quad (2)$$

Here, $\alpha_{t,i}$ represents the attention weight between $\mathbf{q}_t$ and $\mathbf{k}_i$, and d_k is the feature dimension of keys. As sequence length increases, attention computation becomes increasingly dominant in the overall computational cost, presenting significant challenges for long-context processing.

Arithmetic Intensity is the ratio of compute operations to memory accesses. It intrinsically shapes algorithm optimization on hardware. Each GPU has a critical arithmetic intensity determined by its peak compute capability and memory bandwidth, calculated as the ratio of these two hardware limits. For computation tasks, arithmetic intensity above this critical threshold becomes compute-bound (limited by GPU FLOPS), while below it becomes memory-bound (limited by memory bandwidth).

Specifically for causal self-attention mechanism, during training and prefilling phases, batched matrix multiplications and attention computations exhibit high arithmetic intensity, making these stages compute-bound on modern accelerators. In contrast, auto-regressive decoding becomes memory-bandwidth constrained because it generates one token per forward pass while requiring loading the entire key-value cache, resulting in low arithmetic intensity.

This leads to different optimization goals — reducing computation cost during training and prefilling, while reducing memory access during decoding.

3.2. Overall Framework

To leverage the potential of attention with natural sparse pattern, we propose replacing the original key-value pairs $\mathbf{k}_{:t}, \mathbf{v}_{:t}$ in Equation (1) with a more compact and information-dense set of representation key-value pairs $\tilde{K}_t, \tilde{V}_t$ given each query $\mathbf{q}_t$. Specifically, we formally define the optimized attention output as follows:

$$\tilde{K}_t = f_K(\mathbf{q}_t, \mathbf{k}_{:t}, \mathbf{v}_{:t}), \quad \tilde{V}_t = f_V(\mathbf{q}_t, \mathbf{k}_{:t}, \mathbf{v}_{:t}) \quad (3)$$

$$\mathbf{o}_t^* = \text{Attn}(\mathbf{q}_t, \tilde{K}_t, \tilde{V}_t) \quad (4)$$

where $\tilde{K}_t, \tilde{V}_t$ are dynamically constructed based on the current query $\mathbf{q}_t$ and the contextual memory $\mathbf{k}_{:t}, \mathbf{v}_{:t}$. We can design various mapping strategies to get different categories of $\tilde{K}_t^c, \tilde{V}_t^c$, and combine them as follows:

$$\mathbf{o}_t^* = \sum_{c \in C} g_t^c \cdot \text{Attn}(\mathbf{q}_t, \tilde{K}_t^c, \tilde{V}_t^c). \quad (5)$$

As illustrated in Figure 2, NSA have three mapping strategies $C = \{\text{cmp}, \text{slc}, \text{win}\}$, representing compression, selection, and sliding window for keys and values. $g_t^c \in [0, 1]$ is the gate score for corresponding strategy c , derived from input features via an MLP and sigmoid activation. Let N_t denote the total number of remapped keys/values:

$$N_t = \sum_{c \in C} \text{size}[\tilde{K}_t^c]. \quad (6)$$

We maintain a high sparsity ratio by ensuring $N_t \ll t$.

3.3. Algorithm Design

In this subsection, we introduce the design of our remapping strategies f_K and f_V : token compression, token selection, and sliding window.

3.3.1. Token Compression

By aggregating sequential blocks of keys or values into block-level representations, we obtain compressed keys and values that capture the information of the entire block. Formally, the compressed key representation is defined as:

$$\tilde{K}_t^{\text{cmp}} = f_K^{\text{cmp}}(\mathbf{k}_{:t}) = \left\{ \varphi(\mathbf{k}_{id+1:id+l}) \middle| 0 \leq i \leq \left\lfloor \frac{t-l}{d} \right\rfloor \right\} \quad (7)$$

where l is the block length, d is the sliding stride between adjacent blocks, and φ is a learnable MLP with intra-block position encoding to map keys in a block to a single compressed key. $\tilde{K}_t^{\text{cmp}} \in \mathbb{R}^{d_k \times \lfloor \frac{t-l}{d} \rfloor}$ is tensor composed by compression keys. Usually, we adopt $d < l$ to mitigate information fragmentation. An analogous formulation holds for the compressed value representation $\tilde{V}_t^{\text{cmp}}$. Compressed representations capture coarser-grained higher-level semantic information and reduce computational burden of attention.

3.3.2. Token Selection

Using only compressed keys, values might lose important fine-grained information, motivating us to selectively preserve individual keys, values. Below we describe our efficient token selection mechanism that identifies and preserves the most relevant tokens with low computational overhead.

Blockwise Selection. Our selection strategy processes key and value sequences in spacial continuous blocks, motivated by two key factors: hardware efficiency considerations and inherent distribution patterns of attention scores. *Blockwise selection is crucial to achieve efficient computation on modern GPUs.* That is because modern GPU architectures exhibit significantly higher throughput for continuous block accesses compared to random index-based reads. Also, blockwise computation enables optimal utilization of Tensor Cores. This architectural characteristic has established blockwise memory access and computation as a fundamental principle in high-performance attention implementations, as exemplified by FlashAttention’s block-based design. *Blockwise selection follows the inherent distribution patterns of attention scores.* Prior works (Jiang et al., 2024) have shown that attention scores often exhibit spatial continuity, suggesting that neighboring keys tend to share similar importance levels. Our visualization in Section 6.2 also shows this spatial continuous pattern.

To implement blockwise selection, we first divide key, value sequences into selection blocks. To identify the most important blocks for attention computation, we need to assign importance scores to each block. Below we present our method for computing these block-level importance scores.

Importance Score Computation. Computing block importance scores could introduce significant overhead. Fortunately, the attention computation of compression tokens produces intermediate attention scores that we can leverage to induce selection block importance scores, formulated as:

$$\mathbf{p}_t^{\text{cmp}} = \text{Softmax} \left(\mathbf{q}_t^T \tilde{\mathbf{K}}_t^{\text{cmp}} \right), \quad (8)$$

where $\mathbf{p}_t^{\text{cmp}} \in \mathbb{R}^{\lfloor \frac{t-l}{d} \rfloor + 1}$ is the attention scores between q_t and compression keys $\tilde{\mathbf{K}}_t^{\text{cmp}}$. Let l' denote the selection block size. When compression blocks and selection blocks share the same blocking scheme, i.e., $l' = l = d$, we can directly obtain the selection block importance scores $\mathbf{p}_t^{\text{slc}}$ by $\mathbf{p}_t^{\text{slc}} = \mathbf{p}_t^{\text{cmp}}$ straightforwardly. For cases where the blocking schemes differ, we derive the importance scores for selection blocks according to their spatial relationship. Given $l \leq l'$, $d \mid l$ and $d \mid l'$, we have:

$$\mathbf{p}_t^{\text{slc}}[j] = \sum_{m=0}^{l'-1} \sum_{n=0}^{l-d-1} \mathbf{p}_t^{\text{cmp}} \left[\frac{l'}{d}j - m - n \right], \quad (9)$$

where $[\cdot]$ denotes the indexing operator for accessing vector element. For models employing GQA or MQA where key-value caches are shared across query heads, consistent block selection across these heads has to be ensured to minimize KV cache loading during decoding. The shared importance scores across heads in a group are formally defined as:

$$\mathbf{p}_t^{\text{slc}'} = \sum_{h=1}^H \mathbf{p}_t^{\text{slc},(h)}, \quad (10)$$

where (h) in the superscript denotes the head index, and H is the number of query heads in each group. This aggregation ensures consistent block selection across heads within the same group.

Top- n Block Selection. After obtaining the selection block importance scores, We retain tokens within the top- n sparse blocks ranked by block importance scores, formulated as:

$$\mathcal{I}_t = \{i \mid \text{rank}(\mathbf{p}_t^{\text{slc}}[i]) \leq n\} \quad (11)$$

$$\tilde{K}_t^{\text{slc}} = \text{Cat} \left[\{\mathbf{k}_{il'+1:(i+1)l'} \mid i \in \mathcal{I}_t\} \right], \quad (12)$$

where $\text{rank}(\cdot)$ denotes the ranking position in descending order, with $\text{rank} = 1$ corresponding to the highest score, $\mathcal{I}_t$ is the set of selected blocks' indices, Cat denotes the concatenation operation. $\tilde{K}_t^{\text{slc}} \in \mathbb{R}^{d_k \times nl'}$ is tensor composed by compression keys. An analogous formulation applies to the fine-grained value $\tilde{V}_t^{\text{slc}}$. The selected keys and values then participate in the attention computation with $\mathbf{q}_t$ as defined in Equation (5).

3.3.3. Sliding Window

In attention mechanisms, local patterns typically adapt faster and can dominate the learning process, potentially preventing the model from effectively learning from compression and selection tokens. To address this issue, we introduce a dedicated sliding window branch that explicitly handles local context, allowing other branches (compression and selection) to focus on learning their respective features without being shortcut by local patterns. Specifically, we maintain recent tokens $\tilde{K}_t^{\text{win}} = \mathbf{k}_{t-w:t}$, $\tilde{V}_t^{\text{win}} = \mathbf{v}_{t-w:t}$ in a window w , and isolate attention computations of different information sources (compression tokens, and selected tokens, sliding window) into separate branches. These branch outputs are then aggregated through a learned gating mechanism. To further prevent shortcut learning across attention branches with marginal computational overhead, we provide independent keys and values for three branches. This architectural design enables stable learning by preventing gradient interference between local and long-range pattern recognition, while introducing minimal overhead.

After obtaining all three categories of keys and values ($\tilde{K}_t^{\text{cmp}}, \tilde{V}_t^{\text{cmp}}; \tilde{K}_t^{\text{slc}}, \tilde{V}_t^{\text{slc}}; \text{ and } \tilde{K}_t^{\text{win}}, \tilde{V}_t^{\text{win}}$), we compute the final attention output following Equation (5). Together with the compression, selection, and sliding window mechanisms described above, this forms the complete algorithmic framework of NSA.

3.4. Kernel Design

To achieve FlashAttention-level speedup during the training and prefilling, we implement hardware-aligned sparse attention kernels upon Triton. Given MHA is memory-intensive and inefficient for decoding, we focus on architectures with shared KV caches like GQA and MQA following the current state-of-the-art LLMs. While compression and sliding window attention computations are readily compatible with existing FlashAttention-2 kernels, we introduce the specialized kernel design for sparse selection attention. If we were to follow FlashAttention's strategy of loading temporally continuous query blocks into SRAM, it would result in inefficient memory access since queries within a block may require disjoint KV blocks. To address this, our key optimization lies in a different query grouping strategy: for each position on the query sequence, we load all query heads within a GQA group (they share the same sparse KV blocks) into SRAM. Figure 3 illustrates our forward pass implementation. The proposed kernel architecture is characterized by the following key features:

1. **Group-Centric Data Loading.** For each inner loop, load all heads' queries $Q \in \mathbb{R}^{[h, d_k]}$ in the group at position t and their shared sparse key/value block indices $\mathcal{I}_t$.